

Pillai College of Engineering (Autonomous)

New Panvel

Affiliated to University of Mumbai

Department of Computer Engineering

BLOG REPORT

Understanding pyproject.toml in Python

Name : Nair Devika Baburaj

Roll No. : CEA_336

Division : A

Batch : A-2

Understanding pyproject.toml in Python

Python packaging was completely new to me. While working on this topic, I kept seeing files like `setup.py`, `requirements.txt`, `setup.cfg` and `tox.ini`, and I wondered why one project needed so many files. Then I learned about `pyproject.toml`, a single file that replaces most of them. In this blog, I explain what I understood about it.

What is pyproject.toml?

`pyproject.toml` is a configuration file that sits in the main folder of a Python project. It tells Python tools how to build the project, what the project is about, and which libraries it needs.

It is written in **TOML**, which stands for Tom's Obvious, Minimal Language. TOML is easy to read because it only uses key-value pairs and sections (called tables) written in square brackets.

```
[project]
name = "demo"
version = "0.1.0"
```

Here, `[project]` is the section, and `name` and `version` are keys with values. It is that simple.

Why was it needed?

Earlier, Python projects used `setup.py`, which is a Python script. This caused a few problems:

- To find out what a package needed for building, a tool had to run `setup.py`. But `setup.py` itself needed some packages to already be installed. This is a chicken-and-egg problem.
- Project information was spread across many files.
- Every tool had its own config file, which made projects messy.

To fix this, the Python community introduced a few PEPs (Python Enhancement Proposals):

- **PEP 518** introduced `pyproject.toml` and the `[build-system]` section.
- **PEP 517** defined how tools like pip talk to build backends.
- **PEP 621** defined the `[project]` section for storing project details.

The three main sections

1. [build-system]

This section tells pip which tool should build your project.

```
[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"
```

`requires` lists what is needed at build time, and `build-backend` is the tool that does the building. Other popular backends are hatchling, flit-core and poetry-core. We can switch between them by changing just these lines.

2. [project]

This section stores the details of the project, like its name, version and dependencies.

```
[project]
```

```

name = "weather-cli"
version = "0.1.0"
description = "A simple command-line weather tool"
readme = "README.md"
requires-python = ">=3.9"
dependencies = [
    "requests>=2.31",
    "rich>=13.0",
]

```

`dependencies` lists the libraries our project needs. This replaces the old `install_requires` in `setup.py`. `requires-python` makes sure the package is only installed on supported Python versions.

We can also add **optional dependencies**, which are extras that not every user needs, such as testing tools:

```

[project.optional-dependencies]
dev = ["pytest", "black", "ruff"]

```

Then someone can install them using `pip install ".[dev]"`.

We can also create a command-line command using `[project.scripts]`:

```

[project.scripts]
weather = "weather_cli.main:run"

```

After installing the package, typing `weather` in the terminal will run the `run()` function from `weather_cli/main.py`. This is a very useful feature.

3. [tool]

This section is for the settings of other tools. Each tool gets its own sub-section like `[tool.black]` or `[tool.pytest.ini_options]`. This means we don't need separate files like `.flake8` or `pytest.ini`.

```

[tool.black]
line-length = 88

[tool.pytest.ini_options]
testpaths = ["tests"]

```

Tools like Black, Ruff, pytest, mypy and coverage can all read their settings from this file.

A complete example

Here is a small but complete `pyproject.toml`:

```

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "notes-app"
version = "1.0.0"
description = "A tiny note-taking tool"
requires-python = ">=3.10"
dependencies = ["click>=8.1"]

[project.scripts]
notes = "notes_app.cli:main"

```

```
[tool.black]
line-length = 100
```

A simple example

Suppose we create a folder called **hello-app** with just one file, **pyproject.toml**, containing a **[build-system]** section and a **[project]** section like the ones above. When we run **pip install -e .** in that folder, pip reads the file and installs the project. We don't need **setup.py** at all. After that, the project shows up in **pip list**.

This shows the main idea: one file is enough to describe and install a Python project.

Useful commands

Once the file is ready, these commands are enough for most work:

```
# install the project in editable mode
pip install -e .

# install with development extras
pip install -e ".[dev]"

# build the package
pip install build
python -m build
```

The build command creates a **dist/** folder with files that can be uploaded to PyPI using a tool called **twine**.

Modern tools that use it

Many newer tools are built around this file. **Poetry** uses its own **[tool.poetry]** section. **uv** is a very fast package manager that reads the standard **[project]** section. **Hatch** and **PDM** also follow the same standard. Since **[project]** is standardized, we can change tools later without rewriting all our metadata.

Tips I found helpful

- 1 Use the standard **[project]** section instead of tool-specific metadata, so the project works with more tools.
- 2 Always set **requires-python**.
- 3 Use optional dependencies to keep the main install light.
- 4 Keep tool settings under **[tool.*]** so the project folder stays clean.
- 5 Commit the file to Git so everyone builds the project the same way.

Conclusion

pyproject.toml replaces many old and confusing files with one clean, readable file. It holds the build settings, project details, dependencies and tool settings together. After learning it, I feel packaging in Python is much easier to understand. Whether we are publishing a library or just organizing a college project, using **pyproject.toml** makes the work cleaner and easier to share.

This blog is created as part of learning at Pillai College of Engineering (PCE). For more details, visit <https://www.pce.ac.in>.